

Automate Cybersecurity Tasks with Python ×

Module 1 ▼ Introduction to Python

Module 2 ▼ Write effective Python code

Module 3 ▼ Work with strings and lists

Module 4 ▲ Python in practice

Python for automation

- ✓ Welcome to module 4
Video • 1 min
- ✓ Automate cybersecurity tasks with Python
Video • 3 min
- **Automating Tasks in CI/CD**
Reading • 10 min
- Essential Python components for automation
Reading • 8 min
- Clancy: Continual learning and Python
Video • 2 min
- Test your knowledge: Python and automation
Practice Assignment • 8 min

Work with files in Python

Automating Tasks in CI/CD

⏏ 0:03 / 7:55 ● 1x 🔊 ⚙

Automating Tasks in CI/CD: Using Python to Build Security Directly Into Your Pipeline

You've learned how important automation is for security and you've also seen how Python can automate security tasks. Now, let's explore how to use automation, and especially Python, to make your Continuous Integration and Continuous Delivery/Deployment (CI/CD) pipelines more secure.

Just like security analysts use Python to automate security tasks for whole systems, you can use Python to automate security tasks right inside your CI/CD pipeline. This means you build security checks directly into how you build and release software. This is called **DevSecOps**. Think of it as 'Development, Security, and Operations' working together from the beginning. DevSecOps is about making security a shared responsibility and automating security practices as part of your everyday workflow, ensuring that security is considered at every stage of your CI/CD pipeline.

Why Automate Security Tasks in CI/CD with Python?

Imagine manually checking every piece of code for problems, or manually testing every version of software for security issues before it's released. That would be really slow and have a lot of errors. Python is a great tool to automate these security tasks in CI/CD because it's flexible and has many helpful tools - like libraries.

Using Python to automate security tasks in your CI/CD pipeline is beneficial for a few reasons :

- **Increases Speed and Efficiency:** Python scripts for security checks are fast and work well as part of your

